

Hybrid LOD Architecture: Bridging ECS and OOP for Scalable Traffic Simulation in Unity

Abstract

In this paper, I aim to outline the implementation approach and benefits of Unity's DOTS/ECS system. Demonstrating the architecture and systems I created to support a hybrid LOD system utilizing this now finished technology. I will outline its potential by exposing its current use in the industry, comment on why it has not been explored in recent years, then demonstrate how the modern system can be utilized. I will detail my approach alongside the setbacks and my solutions along the way. To further demonstrate its benefits, I will then discuss screenshots of a controlled test scenario visualizing the effects of Pure OOP, ECS, and finally my Hybrid LOD system. I will comment on optimization techniques and practices I implemented alongside beneficial future additions. Then conclude by describing why I believe the technology deserves more investment.

Introduction

Modern open worlds require a level of computational expense that is seen as unachievable in most scopes due to the heavy computing costs of calculating NPC movement behavior using heavy physics such as Raycasts and expensive GC allocation of constantly moving physics transforms. Having Hundreds or indeed thousands of OOP GameObjects wandering around is simply not feasible. The optimization of AI and Physics is often overlooked due to its complexity, as the technology for Graphics Optimization is far better documented and studied. “technology to control the cost of rendering exists in the form of hardware, visibility culling and level-of-detail, whereas few cost control methods have been discussed for physics or AI.” [1].

My proposed solution details bridging the gap between ECS/DOTS and OOP using the Rossetta Stone pattern to manage and index a Twin-Graph Navigation System. The system acts on the proximity of the player's camera. As the player camera nears a vehicle it will Hydrate from its lightweight ECS data form, its data and progress is handed over to the OOP GameObject pulled from pooled memory and Infront of the player is now looking at a fully rendered car driving through the city on its daily business. The opposite then occurs when the player's camera moves away, vanquishing the OOP GameObject and serializing it into contiguous data being where it is a data point being moved kinematically in a burst compiled system.

To the player, nothing is different. It sees the cars in their proximity and watches as they scurry around this big open world city. However, the CPU overhead is drastically reduced, resulting in highly stabilized frame times due to the data it's being handed for the ECS entities in the distance being formatted perfectly for processing.

Related Work and Context

It is understood that in the AAA world of development studios such as Insomniac making the Spider-Man series and CD Projekt Red's Cyberpunk utilize similar systems. While Unreal Engine is largely OOP based other than their Mass Entity for crowd simulation, Unity is the leading body in ECS Technology. Despite this, it is largely avoided due to the fact that Unity's DOTS API changed constantly for five years while in preview. Thus, making it notorious for breaking projects and introducing severe technical debt and instability for developers.

My goal is to document its use case in Unity 6 where it is finally production ready.

Architecture and Methodology

The Rosetta Stone

The initial hurdle to conquer when making a city-based traffic system or AI NPC cars is of course the road. Everything needs to know where the road is and how to navigate it. This created an issue as the road that the ECS entities base on their navigation cannot exist in the OOP world and vice versa. Therefore we need two identical roads, copies of one another. This way both systems have their own copy of the road and can possess the capabilities of navigating it based on the same map of the city. From this another small issue arises. What if you tweak one of the roads, then the other system is now navigating a different city. ECS may as well be in Los Angeles while OOP is in New York. Thus, I designed the Waypoint Sync tool. It takes the OOP Waypoint Network to transform as its base and then applies those changes to the ECS network that lives in the DOTS Subscene. As each node stores possible visitable nodes from that node, this has the potential to introduce fatal pathfinding desynchronization. This lends quick iteration and tweaks saving from level designing headaches. Another tool that was imperative to rapid iteration was the Make Roads Two-way tool, as the node-based network I designed logically operated as a one-way system in code. This meant that the CarAI could be given a destination node it logically couldn't reach from its current node.

The main challenge of the Hydration and Dehydration of the respective systems vehicles was the data handover. Hence, the Rosetta Stone Pattern. The most universal language in all of computing is the integer, and utilizing this common tongue is what allowed these systems to communicate. Each node in both the DOTS subscene and the OOP overworld were given unique integer identifiers that are exactly correlated. Meaning that Node_7 in the OOP network possessed the exact same identifier as Node_7 in the DOTS subscene network. Using this technique allows the ECS and OOP vehicles to simply package the data in their buffer (where they are and where they are going) into integers and pass them along. The other vehicle receiving this data could instantly apply it to its pathfinding and off it goes, seamlessly.

Quest Trigger

Naturally, this system is not isolated in this game world. Furthermore, mimicking the workflow of a large game studio system can be engineered by various developers. A matter of large importance for the project was dependency inversion, ensuring that any dependencies were injected into other systems potentially realized by other team members. My CarAI can be plugged into any Car model and driving physics, and their pathfinding can be allocated by any quest dispatching system. Once the Quest Trigger has been given one or many destination nodes for one or many AI vehicles, it acts as a heartbeat of data. Every eight seconds the global event bus retrieves the data from the Quest trigger and delivers it to OOP and ECS cars alike. Regardless of whether the vehicle is in an ECS or OOP state, they will independently calculate their own A* path to their desired location. This in turn means that we end up with a Skyrim system where NPCs do not rely on simple coordinate translation (teleportation) to their desired location but travel in real time. If the player discusses a meeting location and time with an NPCs they can be seen driving every mile of the way there, and more importantly will do regardless of whether or not you can see them.

Hydrated OOP AI Brain

In terms of immersion and game design, this is the most important system in the project. This is what the player actually sees touches and feels. When the player's camera is in range, we get a high-fidelity simulation of traffic or race cars depending on the context.

I designed a component-based system that breaks down every part of the moving engine that is an AI brain and curated a struct data stream of information firing between them every frame. The AI Brain holds all the information and orchestrates the operation. It passes the driver's profile data from the scriptable object plugged into the Car prefab, giving it its personality traits like patience and driving style. It passes this data to the AI Sensor Module (the cars eyes) as it contains vital data such as the length of the ray cast (to simulate how cautiously and at what speed the driver operates at). The Sensor Module returns its findings back to the brain after looking at the world in the form of a struct (Upcoming obstacles, traffic lights etc). The Brain then relays this data, again alongside the driver's profile data to the AI Decision module. The stateless Decision module reads the data runs it through its algorithm and concludes in the form of an AI Command (another struct) and returns it to the Brain. Finally, the Brain dispatches its final calculated command to the IVehicleInput Interface simply in the form of desired speed, steering and whether the handbrake is on. Then whatever physics system is plugged in reads these three variables and applies it to the engine and off it goes.

The elegance really comes from the dependency inversion here. The AI never touches the car physics. It outputs data through the aforementioned interface which in turn is read by a car physics system.

The Unmanaged Domain

The fundamental philosophy driving the ECS architecture is Data-Oriented Design (DOD). Mike Acton, former Engine Director at Insomniac Games, famously stated regarding game optimization: *"Where there is one, there are many. If you have one of something, you probably have a lot of them... and you should*

process them together." [2] By stripping away the OOP GameObjects and organizing our vehicle data into contiguous arrays, the Burst Compiler can process the movement of thousands of entities in a single, cache-friendly CPU instruction.

Well, what happens when the camera is far away and there's no chance of the player seeing us? How do we move a node of data in a physical world. How does a row on a spreadsheet (archetype) drive on a road?

The DOTS Baker strips all OOP data, turning our heavy GameObject into a contiguous string of data. It is reduced to three components. Its tag, its simulation data holding its speed and destination and lastly the dynamic buffer holding the remaining A* path. This seamless data transfer is only possible via the Rosetta Stone. While the remaining A* path is transferred during Hydration and Dehydration, the respective cars do independently calculate their paths using their own A* system. We pass it over to save computing something that has already been calculated, saving performance.

The most performant system on the ECS side is its burst compiled movement system running on its own CPU thread. As the ECS data point is not a physical object in a real world, it is moved kinematically using pure math's adjacent to the main thread. This allows us to scale up to thousands of entities if we so chose, and its performance cost would still be negligible.

The Baton Passing

OOP -> ECS (Dehydration)

When an OOP vehicle goes beyond the dehydration radius, the LOD Manager reads the data it needs from the vehicle to pass to the ECS vehicle. Previously mentioned data such as its position, remaining A* route etc. This is read from the cars GPS system the AI Navigator which functions acting as a localized navigation module. The ECS Entity is created at the exact same position, and the mention data is injected into it, i.e. where to go and how to get there. This is stored in the entity's Dynamic Buffer which in turn keeps the data in the event of need to hand the baton back over.

ECS -> OOP (Hydration)

The process here is much the same except we are spawning in a heavier GameObject. Again, the LOD manager reads the Entities data puts it in the baton but before passing it over, we need to pull the prefab from pooled memory. The prefab is pulled from memory saving any GC spikes as all the OOP cars are pooled, and the Data is injected directly into the newly instantiated OOP navigation. Once again, the car is created at the exact point the entity left off, and the entity itself is destroyed.

Implementation Details and Optimization

Zero GC Object Pooling

As previously mentioned, OOP GameObjects are quite heavy and thus require optimization techniques and practices. The most important thing was pooling more cars in memory than we would ever need to spawn. The Vehicle Pool on startup warms up a specified amount of memory to store our OOP cars (in this case 150 cars). They are placed in memory and then disabled, waiting for use to save the computational expense of potentially instantiating hundreds of Game Objects on the fly. Simply all we must do from this point is toggle Set Active to true or false based on our need for them or lack thereof.

Domain Isolation

A trivial issue I encountered during development was the fragility of having these two parallel universes co-existing. Due to the fact the DOTS Sub Scene and OOP scenes existed together in the editor using Find Object By Type was unintentionally retrieving DOTS Nodes and handing them over to the OOP A* pathfinding algorithm. This of course causes A* to fail as it cannot map routes across dimensions. The solution was to simply filter the reading of Nodes in the Waypoint Networks awake to ignore any nodes not belonging to the OOP Game World. Thus, ensuring the A* algorithm in AI Pathfinder was only ever handed nodes belonging to routes on the physical graph.

Results and Performance Analysis

Test Methodology: 100 vehicles spawned in a continuous dispatch loop, performance monitored using the Unity Profiler.

Scenario 1: Pure OOP

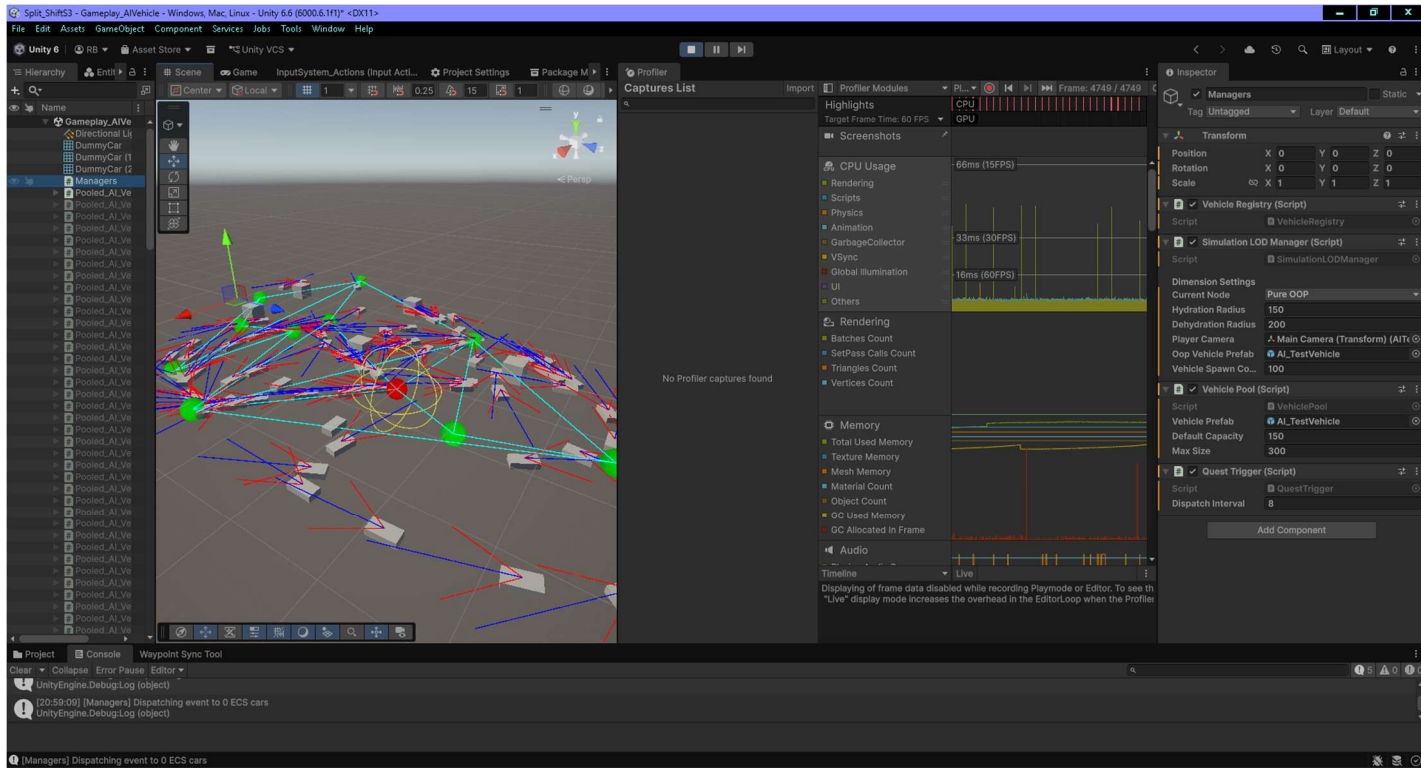


Figure 1: The Twin-Graph Navigation System. Physical WaypointNodes (green) are mirrored into a DOTS SubScene, utilizing deterministic integer hashing (The Rosetta Stone pattern) to ensure parity across dimensions.

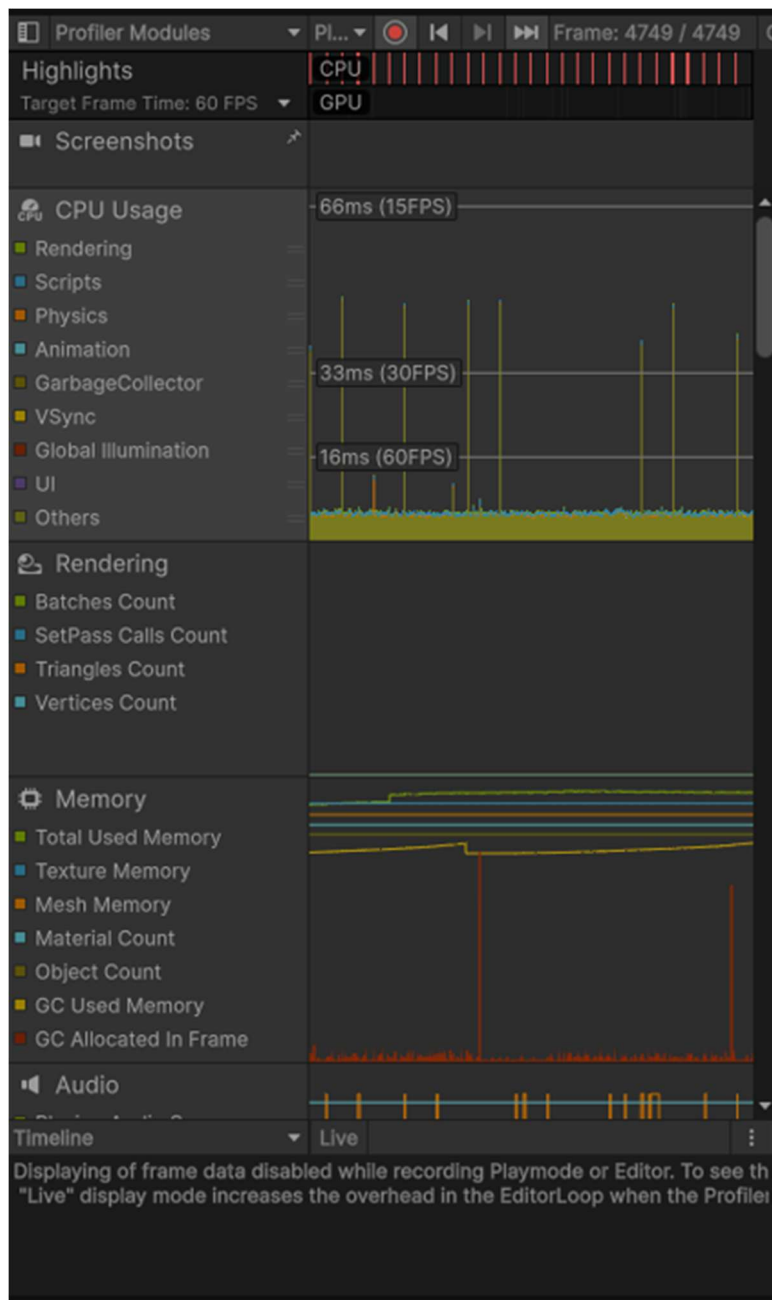


Figure 2: Unity Profiler capture of the Pure OOP simulation. Note the severe Garbage Collection spikes (orange line) and high CPU frame times resulting from the overhead of 100 active MonoBehaviours and physics Rigidbodies.

As detailed in the screenshot, we can observe the stress that the OOP GameObjects put the CPU under. Each of the 100 Objects have Rigidbodies, Update loops and Ray casts all of which must be processed by the CPU every single frame. Not to mention the fact that they are all stored on the heap slowing down the retrieval of their data in the first place. This architecture is simply not viable due to the bottleneck being created by the CPU.

Scenario 2: Pure ECS

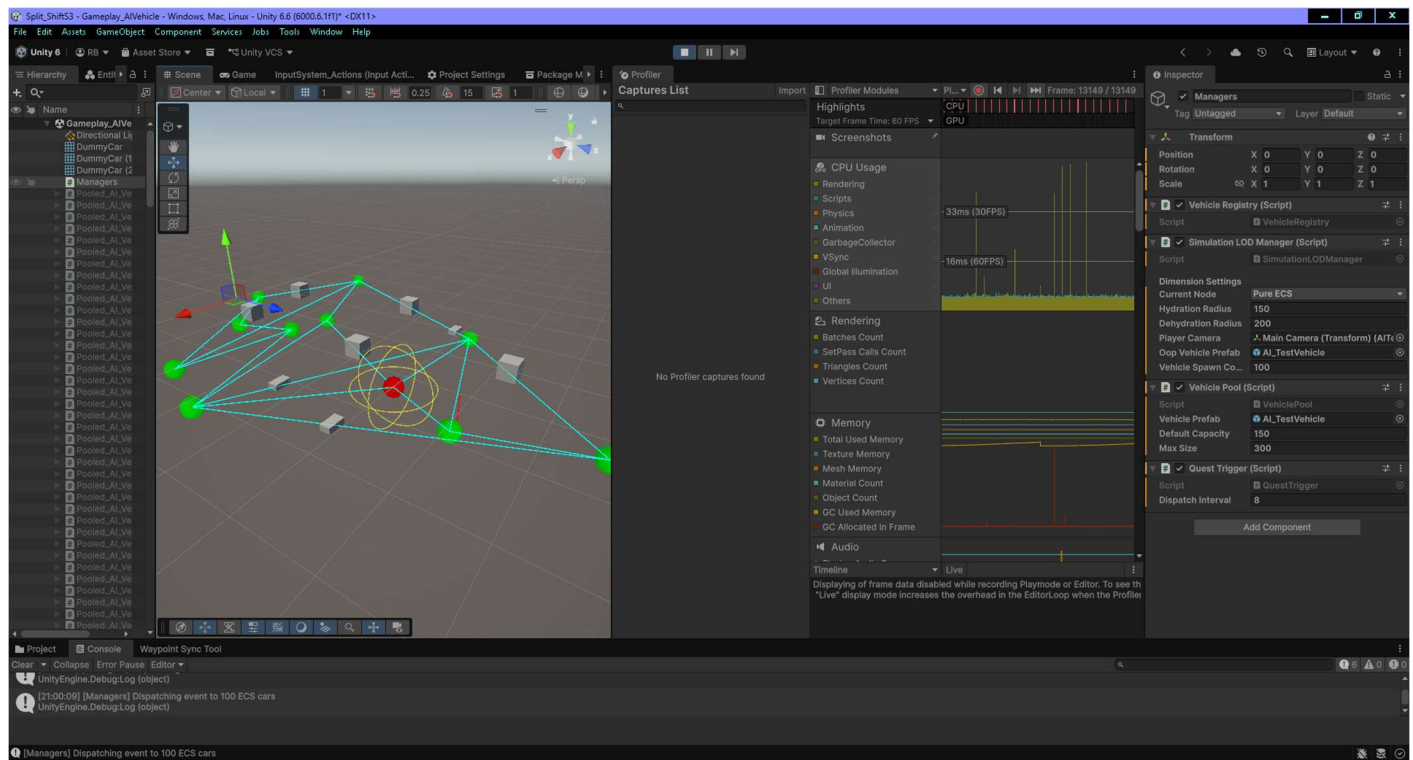


Figure 3: Scene View visualization of the Unmanaged Domain. Distant vehicles are dehydrated into pure ECS entities, represented by red debug poles, navigating the deterministic node graph.

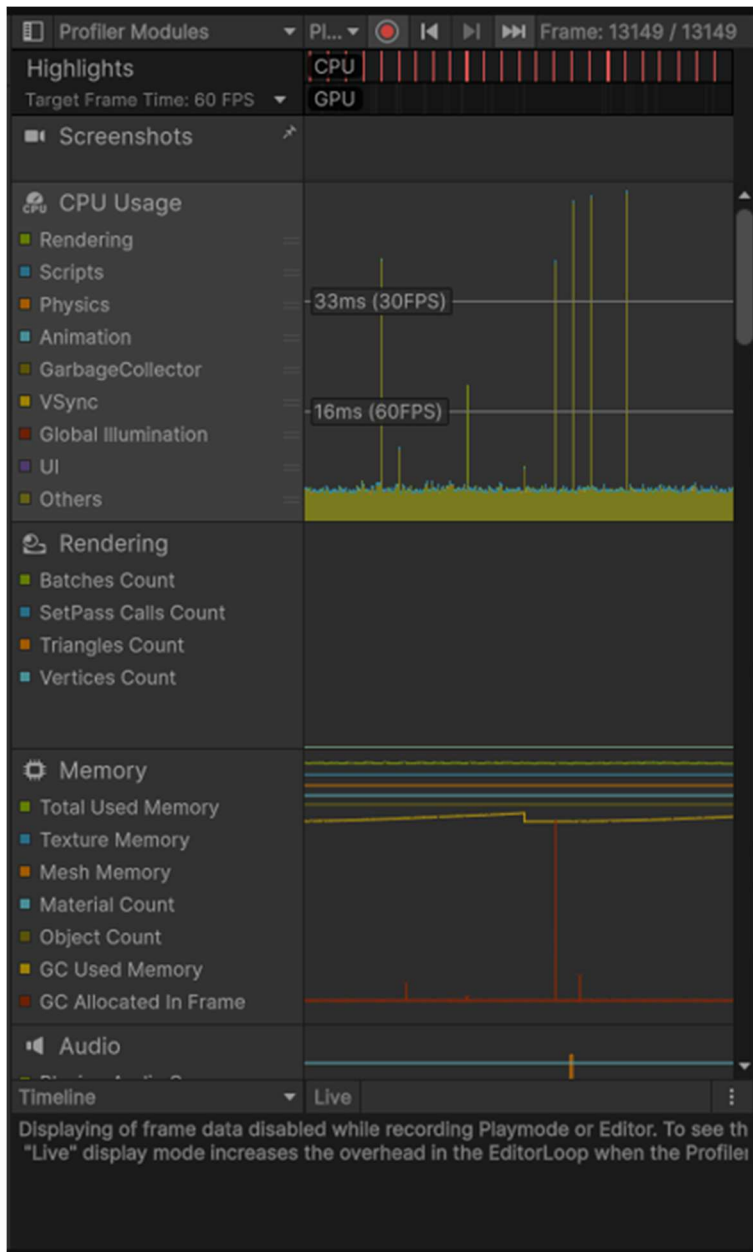


Figure 4: Unity Profiler captures the Pure ECS simulation. The CPU frame time is drastically reduced, and Garbage Collection is eliminated, demonstrating the efficiency of Burst-compiled kinematic mathematics and contiguous memory arrays.

Notice the more stable CPU graph. By converting the 100 cars into contiguous memory arrays processed by the burst compiler, we greatly reduce the stress on the main thread. Unfortunately, we cannot play a game with just invisible data points navigating the background simulation, therefore the hybrid system gives us the best of both worlds. Lightning-fast data processing alongside the high-fidelity traffic and car simulations.

Scenario 3: Hybrid LOD

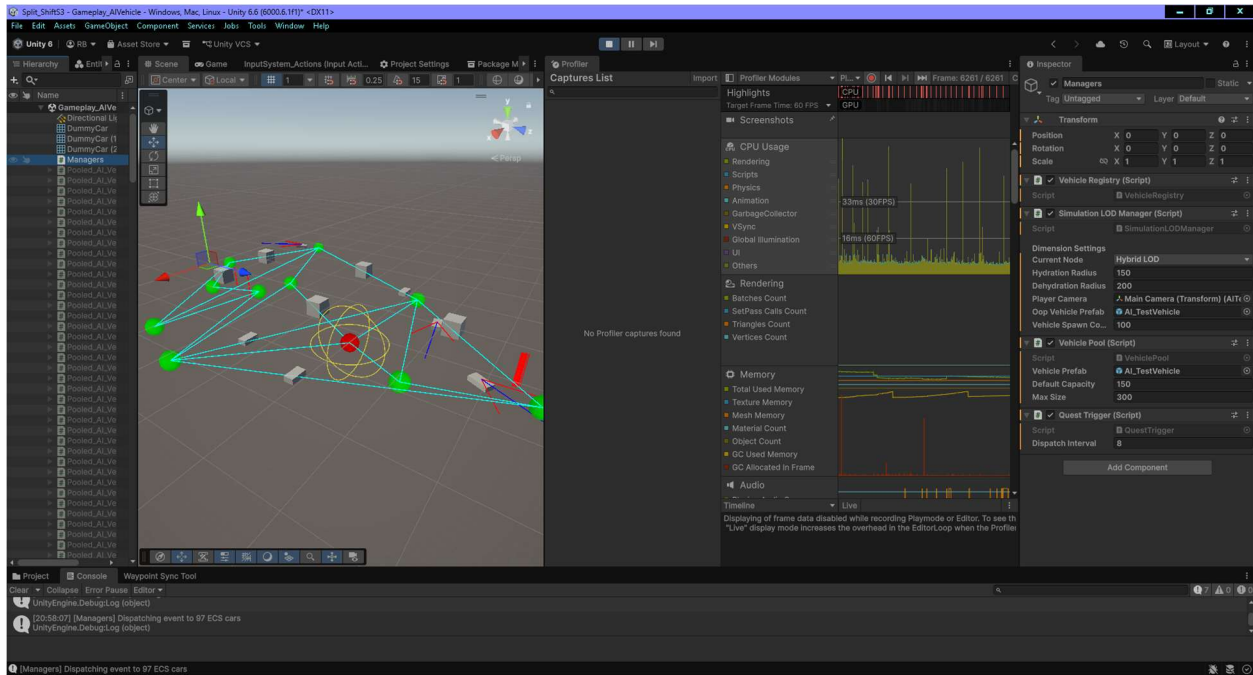


Figure 5: Custom Unity Editor tooling developed to automatically synchronize and remap connections between the Managed (OOP) and Unmanaged (DOTS) waypoint networks, preventing cross-dimensional pathfinding failures.

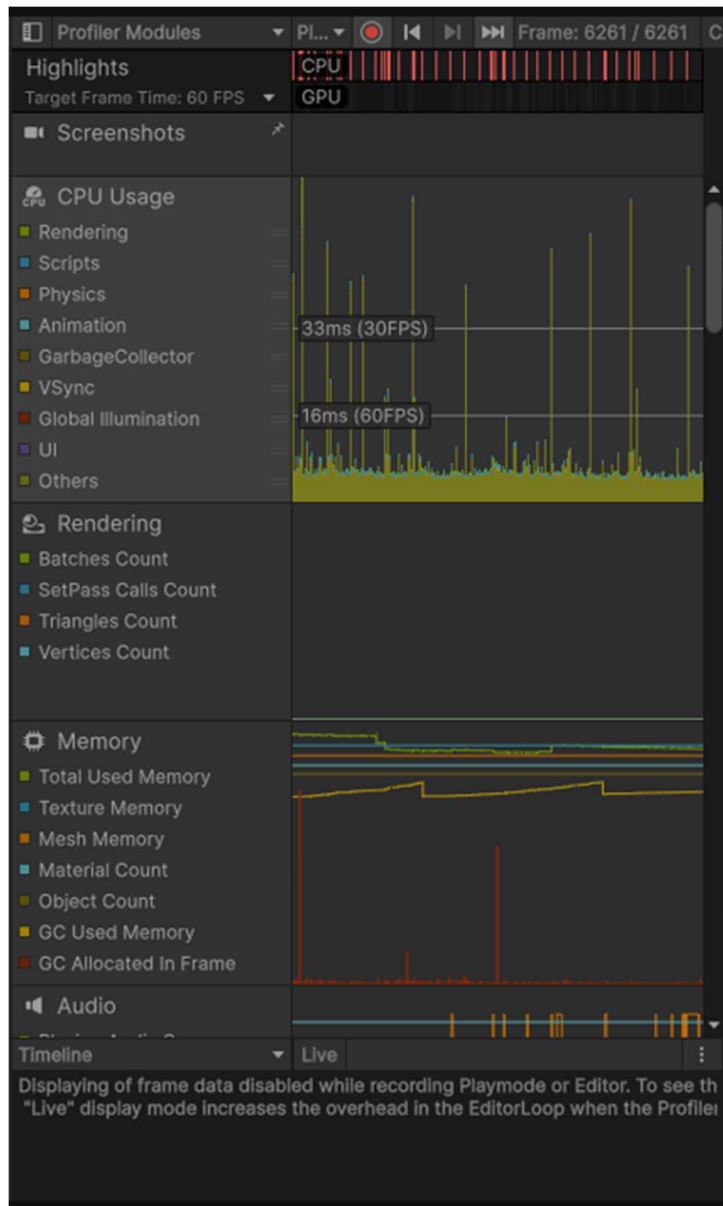


Figure 6: Unity Profiler capture of the Hybrid LOD architecture. The system dynamically balances computational load, maintaining stable frame times while preserving high-fidelity simulation within the player's immediate proximity.

Performance statistics here are naturally arbitrary of course as they will differ depending on where the player is. Moreover, where the player's camera is and what's inside or outside the hydration radius. We can see here there is a balance of both ECS entities indicated by the rendered Red Poles I put in for debugging purposes and fully realized OOP cars. The idea that largely makes this system beneficial is the fact that we are leaving no performance on the table. Sometimes we simply must simulate several OOP

cars to simulate immersive traffic, however only when it is absolutely necessary. This ensures that we are leaving no performance on the table.

Future Developments

Naturally, of course, there is still persistence in the limitations of ECS systems. ECS entities at present have no avoidance unlike the intelligent movement of the OOP cars. Furthermore, future progression of the system would feature a burst complied algorithm dedicated to the separation of the flocking ECS nodes to further simulate the touch and go nature of traffic. This would again go towards scraping together more performance as we avoid expensive physics collider and ray cast calculations occurring in every frame.

Conclusion

I have outlined a way in which the now finished ECS/DOTS system can be integrated into AAA and indie development workflows alike. While I do acknowledge its complexity and drawbacks, I believe it is a necessary hurdle to jump. Otherwise, massive amounts of performance could be left sitting idle as your game stutters its way to failure. With the growing complaint of performance issues amongst the Gaming community, I cannot overstate its importance to further our exploration and education on such systems. A perfect example lies in the initial and secondary release of Cyberpunk. The initial release was unplayable, resulting in refunds to all players and massively setting the studio back. However, the project they shipped upon their second release featured a LOD system very similar to this one thus demonstrating the effect investments into this technology can have.

References

- [1] S. Chenney, "Simulation level-of-detail," in *Proceedings of the Game Developers Conference*, San Jose, CA, 2001, pp. 45-50.
- [2] M. Acton, "Data-Oriented Design and C++," presented at *CppCon*, Bellevue, WA, Sept. 2014. [Online].